# DIGIGUARD & CEREBRO WEB4 EVRENSEL MASTER ANAYASASI

*CİLT 2: SOPON KENDİ KENDİNİ ONARMA (SELF-HEALING) TEKNOLOJİSİ, DEAI AJAN YAPILARI VE KATMAN-2 SHARDING PROTOKOLLERİ*

---

**Sürüm Kontrolü:** v3.2 (Tam Otonom Kalıcılık Standardı)

**Odak Katmanı:** Kendi Kendini Yaşatan Kontrat Mimarisi, Merkeziyetsiz Yapay Zekâ Senkronizasyonu, Milyarlarca Ürün İçin Merkle Tree ve Sharding Ölçeklenebilirliği

**Yürürlük ve Sertifikasyon Otoritesi:** Cerebro Web4 Core Protokolü (Anti-Fragile Mod)

# BÖLÜM I: SOPON TEKNOLOJISI VE KENDI KENDINI ONARMA (SELF-HEALING) PROTOKOLÜ

## 1.1 İnsan Müdahalesiz Sürekli Kalıcılık Esası

**Madde 1 –** Sopon Protokolü, Digiguard ve Cerebro ekosisteminin dış kurumsal yapılardan, fiziksel sunucu merkezlerinden veya insan mühendislerin varlığından bağımsız olarak sonsuza kadar kendi kendini yaşatabilmesini garanti eden otonom motor katmanıdır. Web4 mimarisi üzerinde çalışan bu teknoloji, ağ üzerindeki akıllı sözleşmelerin statik birer kod bloku olmaktan çıkıp, dış dünyadaki değişimleri otonom şekilde algılayan ve kendi kod bütünlüğünü koruyan canlı yapılar olarak çalışmasını şart koşar. Herhangi bir şirketin tasfiyesi, devlet müdahalesi veya küresel sunucu kesintileri sistemin çalışmasını durduramaz.

## 1.2 Kendi Kendini Onarma (Self-Healing) ve Durum Güncelleme Mekanizması

**Madde 2 –** Sistem mimarisinde meydana gelebilecek olası bir veri bozulması, düğüm (node) kayıpları veya arayüz senkronizasyon hataları durumunda Sopon motoru otonom olarak devreye girer. Merkeziyetsiz Oracle ağları üzerinden gelen çapraz doğrulamaları kullanan motor, bozulan veri segmentini izole eder ve yedek blockchain kanalları üzerinden sistemi mikrosaniyeler içinde onarır. Kullanıcılar veya kurumsal entegrasyonlar bu onarım sürecinde hiçbir kesinti, donma veya yanıt vermeme durumuyla karşılaşmazlar; sistem kesintisiz olarak çalışmaya ve sertifikasyon süreçlerini yürütmeye devam eder.

# BÖLÜM II: MERKEZIYETSIZ YAPAY ZEKÂ (DEAI) AJAN YAPILARI VE ANOMALI TAKIBI

## 2.1 Otonom Yapay Zekâ Ajanlarının Görev Tanımları

**Madde 3 –** Cerebro ağının güvenliği ve veri doğruluğu, zincir üstünde (on-chain) yaşayan ve insan denetimine tabi olmayan Merkeziyetsiz Yapay Zekâ (DeAI) ajanları tarafından sağlanır. Bu ajanlar, sinirsel ağ mimarileriyle doğrudan akıllı sözleşmelere entegre edilmiştir. Temel görevleri; kayıt-kabul anından itibaren tüm ekosistemdeki veri akışını, biyometrik eşleşmeleri ve finansal transferleri gerçek zamanlı olarak taramak, sahtecilik ve anomali girişimlerini anında bertaraf etmektir.

### 2.2 Gerçek Zamanlı Anomali Tespiti ve Kriptografik Reaksiyon

**Madde 4 –** DeAI ajanları, sisteme akan milyarlarca veride en ufak bir yapısal sapma veya mantıksal uyumsuzluk algıladığı an, ilgili işlemi saniyeler içinde askıya alma yetkisine sahiptir. Bu süreç şu kriptografik reaksiyon adımlarıyla yürütülür: * **a) Sarı Kod Reaksiyonu (Sopon Senkronizasyonu):** Veri akışında geçici bir ağ gecikmesi veya eksik veri paketleri algılandığında, sistem sarı kod moduna geçer. Sopon motoru otomatik olarak kayıp paketleri oracle hatları üzerinden tamamlar ve arayüzde 'Sistem Güncelleniyor' grafiğini canlı tutar. * **b) Kırmızı Kod Reaksiyonu (Anomali Engelleme):** Bilerek manipüle edilmiş bir SHA-256 hashi veya sahte KYC verisi algılandığında sistem kırmızı alarm durumuna geçer. DeAI ajanı, ilgili cüzdan adresini ve Cerebro seri numarasını dondurur, "Sertifika Oluştur" fonksiyonunun tetiklenmesini yazılımsal olarak kalıcı olarak engeller.

## BÖLÜM III: KATMAN-2 SHARDING, MERKLE TREE YAPISI VE MILYARLARCA ÜRÜN ÖLÇEKLENEBILIRLIĞI

### 3.1 Saniyede Milyonlarca İşlem ve Merkle Tree Entegrasyonu

**Madde 5 –** Küresel imalat endüstrilerinde üretilen milyarlarca cansız ürünün ve ticari emtianın aynı anda tescil edilmesi sırasında ana blockchain ağının kilitlenmesini veya fahiş işlem ücretlerinin oluşmasını engellemek amacıyla **Katman-2 (Layer-2) Sharding** mimarisi zorunludur. Üretim bantlarından gelen milyonlarca ürün verisi, alt ağlarda Merkle Tree (Özet Ağaçları) yapısı kullanılarak hiyerarşik olarak gruplandırılır. Milyonlarca ürünün tüm meta-verileri tek bir kök özet (Merkle Root) haline getirilerek ana blockchain omurgasına tek bir işlem (transaction) olarak mühürlenir.

### 3.2 Veri Ağaçları ve Otonom Durum Kanalları (State Channels)

**Madde 6 –** Ürünlerin ve canlıların durum değişiklikleri, sürekli olarak ana ağa yazılmak yerine otonom durum kanalları (State Channels) üzerinde anlık olarak işlenir. Bir ürünün lojistik transferleri, el değiştirmeleri veya bir canlının sağlık durumu verileri bu kanallarda birikir. Belirli periyotlarla veya kritik bir eşik aşıldığında, Sopon motoru durum kanallarındaki son neticeyi tek bir kriptografik kanıt (Proof) halinde ana ağa nakşeder. Bu sayede sistem sonsuz bir ölçeklenebilirlik potansiyeline kavuşur.

# BÖLÜM IV: OTONOM PUKO DÖNGÜSÜ VE YAZILIMSAL ANTI-KIRILGANLIK

## 4.1 Yapay Zekâ Güdümlü Sürekli İyileştirme Mekanizması

**Madde 7 –** Sistemin yazılımsal optimizasyonu, siber savunma güncellemeleri ve akıllı sözleşme yenileme süreçleri, geleneksel yazılım şirketlerinin haftalar süren geliştirme döngülerinden tamamen çıkarılmıştır. Sistem, otonom PUKO (Planla - Uygula - Kontrol Et - Önlem Al) döngüsünü kendi içinde bir kapalı devre motor olarak çalıştırır. DeAI ajanları, ağ genelinde siber tehdit veya performans darboğazı analizleri yapar.

## 4.2 Testnet Simülasyonu ve Canlı Ağ (Mainnet) Güncelleme Şartları

**Madde 8 –** Otonom PUKO döngüsünün işleyiş adımları şu şekildedir: 1. **Planla:** Algoritmik verimlilik veya siber güvenlik yaması için optimizasyon kodu otonom olarak tasarlanır. 2. **Uygula:** Hazırlanan yeni kod bloku, izole bir Web4 Test Ağında (Testnet) yapay zekâ tarafından simüle edilir ve milyonlarca stres testine tabi tutulur. 3. **Kontrol Et:** Yeni kodun sıfır hata ile çalıştığı ve geriye dönük veri bütünlüğünü bozmadığı matematiksel kanıtlarla doğrulanır. 4. **Önlem Al:** Onaylanan yama, canlı akıllı sözleşmelere (Mainnet) insan müdahalesi ve sistem kesintisi olmaksızın otonom olarak enjekte edilir.

# BÖLÜM V: TEKNIK EKIP KODLAMA DIREKTIFLERI VE SOPON SELF-HEALING KONTRAT ŞEMASI

## 5.1 Geliştiriciler İçin Kendi Kendini Güncelleyen Kod Standartları

**Madde 9 –** Yazılım geliştiriciler, yazacakları tüm alt modüllerde proxy kontrat mimarilerini ve yükseltilebilir (upgradable) akıllı sözleşme kalıplarını kullanmak zorundadır. Ancak bu yükseltme yetkisi (Admin Role) kesinlikle bir insana veya kurucu heyete verilemez; yetki doğrudan DeAI kontrat adresine mühürlenmelidir. Veri tabloları ve değişken şemaları dinamik genişlemeye uygun tasarlanmalıdır.

## 5.2 Sopon & DeAI Entegreli Kendi Kendini Onaran Kontrat Kodu

**Madde 10 –** Aşağıdaki kod bloku, sistemde oluşabilecek anomalileri takip eden, durum kanallarından gelen Merkle Root girdilerini mühürleyen ve otonom PUKO güncellemelerini yürüten ana Sopon çekirdek mimarisidir:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title CerebroSoponEngine
 * @dev Digiguard & Cerebro Cilt 2 Sopon Motoru ve DeAI Anomali Kontratı
 */
contract CerebroSoponEngine {

    enum SystemStatus { SecureGreen, UpdatingYellow, AlertRed }
    SystemStatus public currentSystemState;

    address public otonomDeAIAgent;
    bytes32 public currentMerkleRoot;
    uint256 public totalShardsProcessed;

    struct OptimizationPatch {
        bytes32 patchCodeHash;
        bool simulationPassed;
        bool deployed;
    }

    mapping(uint256 => OptimizationPatch) public patchLog;
    uint256 public patchCount;

    event SystemStateChanged(SystemStatus indexed previousState, SystemStatus indexed newState);
    event ShardMerkleRootCommitted(bytes32 indexed merkleRoot, uint256 shardIndex);
    event OtonomPatchExecuted(uint256 indexed patchId, bytes32 patchHash);

    modifier onlyDeAI() {
        require(msg.sender == otonomDeAIAgent, "SOPON-ERR-01: Sadece DeAI Ajanı bu motoru tetikleyebil
        _;
    }

    constructor(address _deAi) {
        otonomDeAIAgent = _deAi;
        currentSystemState = SystemStatus.SecureGreen;
    }

    function setSystemState(SystemStatus _newState) external onlyDeAI {
        SystemStatus oldState = currentSystemState;
        currentSystemState = _newState;
        emit SystemStateChanged(oldState, _newState);
    }

    function commitShardMerkleRoot(bytes32 _merkleRoot) external onlyDeAI {
        require(currentSystemState != SystemStatus.AlertRed, "SOPON-ERR-02: Sistem kırmızı alarmda, ve
        currentMerkleRoot = _merkleRoot;
        totalShardsProcessed++;
        emit ShardMerkleRootCommitted(_merkleRoot, totalShardsProcessed);
    }

    function executeOtonomPukoPatch(bytes32 _patchHash, bool _simResult) external onlyDeAI {
        require(_simResult, "SOPON-ERR-03: Simülasyon testlerinden geçemeyen yama uygulanamaz");
```

```
        patchCount++;
        patchLog[patchCount] = OptimizationPatch({
            patchCodeHash: _patchHash,
            simulationPassed: true,
            deployed: true
        });

        emit OtonomPatchExecuted(patchCount, _patchHash);
    }

    function isSystemOperational() external view returns (bool) {
        return (currentSystemState != SystemStatus.AlertRed);
    }
}
```

**Madde 11 –** Cilt 2 bünyesinde saptanan otonomizm kuralları, Sopon motor işleyiş esasları, Merkle Tree veri birleştirme algoritmaları ve yapay zekâ güdümlü PUKO döngüleri, sistemin anti-kırılgan can damarıdır. Bu cildin kuralları, ağın kendi kendini modifiye etme yeteneği ile birlikte tüzüğe koparılamaz bir zincirle bağlanmıştır.